

A Unified Optimization and Hybrid Learning Framework for Cold-Start Mitigation in Serverless Computing

Sandeep Nekkala^{1*}, Sandeep Kumar Dash^{1†} and Vinay Raj ^{2†}

^{1*}Department of Computer Science & Engineering, NIT Mizoram,
Aizawl, 796012, Mizoram, India.

²Department of Computer Applications, NIT Tiruchirappalli,
Tiruchirappalli, 620015, Tamil Nadu, India.

*Corresponding author(s). E-mail(s): dt22cs003@nitmz.ac.in;
Contributing authors: sandeep.cse@nitmz.ac.in; vinayraj@nitt.edu;

[†]These authors contributed equally to this work.

Abstract

Serverless computing has emerged as a transformative cloud computing paradigm by allowing developers to focus on application logic and abstracting the infrastructure management. However, serverless platforms suffer from a major challenge called as cold start caused by the delays in container initialization violating the service level agreements. Existing mitigation techniques such as prewarming and prior workload prediction costs excessive resource overhead and also fail to adapt to dynamic workload variations. To overcome the limitations of the existing solutions, a unified cold start optimization model under stochastic request arrivals is proposed in this paper by formulating the mitigation strategy as weighted optimization problem. Additionally, to make the optimization model more effective under dynamic workloads, a novel hybrid transformer-reinforcement learning scheduler approach is proposed that integrates long range work load prediction. To further validate the approach, experimental evaluation using real-world workload log traces is performed using AWS Lambda functions. The results demonstrate that the proposed approach significantly reduces cold start latency with better latency-cost-energy tradeoffs.

Keywords: Serverless computing, cold start latency, optimization, reinforcement learning.

1 Introduction

Serverless computing has become a revolutionary paradigm in cloud computing with its pay-per-use economic model, dynamic scalability, and abstraction of infrastructure management, allowing developers to concentrate on application logic instead of server provisioning [1, 2]. The schematic diagram of serverless computing is depicted in Figure 1 which shows the different components in a typical system. Serverless architectures, which are mainly realized through Function-as-a-Service (FaaS) platforms like AWS Lambda, OpenWhisk, and Knative, perform functions in response to events and automatically scale resources to meet demand [3]. Despite these benefits, the effectiveness of serverless platform is hampered by a crucial performance bottleneck called the "cold start" problem, especially for latency-sensitive applications like interactive web services, IoT systems, and real-time analytics [4, 5]. The latency is introduced when a function is called and the corresponding runtime environment needs to be initialised after a period of inactivity. The delay cause due to cold start problem can vary from milliseconds to few seconds [6, 7]. The need for efficient mitigation techniques is highlighted by the fact that these delays impair user experience and jeopardize service-level agreements (SLAs) [8].

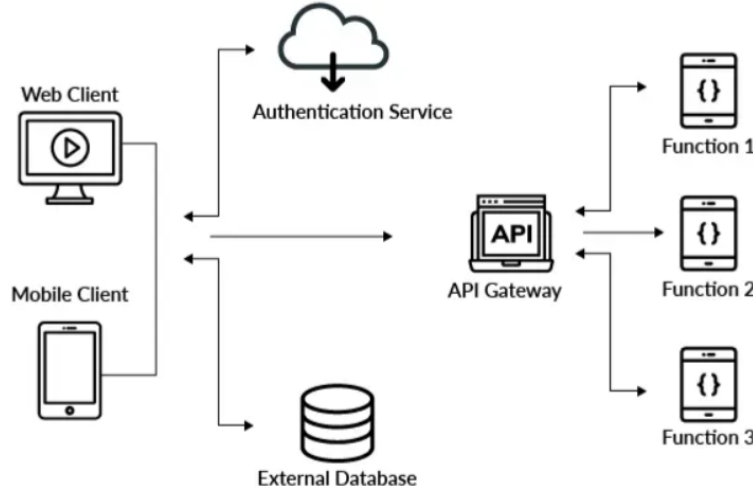


Fig. 1: Serverless Schematic

To mitigate cold-start latency, both academic researchers and industry practitioners have developed diverse approaches, each focusing on a distinct aspect of the initialization process. Pre-warming techniques preserve running virtual machines or containers to prevent initialization overheads, greatly lowering latency but frequently at the expense of higher resource usage [9]. For instance, in the work done by the authors in [10] shows that predictive pre-warming, in which containers

are initialized based on expected workloads, can reduce cold start frequency by up to 60%. By keeping pools of pre-warmed containers, container pooling expands this idea and can reduce the 99th percentile response time by up to 85%. However, it comes with trade-offs in memory usage and operating costs [11]. In workflow-driven applications in particular, function fusion reduces cold start delays and minimizes redundant initializations by combining several functions into a single execution unit [12]. However, the generalizability of the method is limited because it works best when workflow structures are known.

Advanced methods use artificial intelligence (AI) and machine learning (ML) to optimize resource allocation and predict invocation patterns. In comparison to static approaches, reinforcement learning (RL) and long short-term memory (LSTM) models dynamically modify container pre-warming and scaling policies, resulting in 12.73% less memory consumption and 22.65% more pre-warmed invocations [13]. Comparing speculative provisioning to Knative, it can reduce platform overheads by up to 18 times by anticipating function chains in workflows [14]. Although these AI-driven techniques are excellent at adjusting to changing workloads, they may have trouble generalizing across various platforms and necessitate complicated training [15]. By storing function states or allocating resources at the moment of invocation, snapshotting and just-in-time provisioning provide alternate solutions; however, they frequently struggle to manage erratic traffic spikes [16].

While numerous studies have investigated cold-start behavior through empirical benchmarking and platform-specific measurements, the problem has largely been treated as an implementation-level performance issue rather than a formally defined optimization challenge. Most existing approaches either rely on heuristic prewarming strategies or apply machine learning models for workload prediction without explicitly integrating the underlying system dynamics, cost structure, and energy implications into a unified decision framework. As a result, cold-start mitigation remains predominantly reactive and platform-dependent, lacking theoretical grounding and generalizable optimization principles.

In reality, cold-start mitigation is not merely a latency reduction problem. It is fundamentally a stochastic resource allocation problem characterized by uncertain request arrivals, provider-imposed container lifecycle policies, and multi-dimensional trade-offs between performance, operational cost, and energy consumption [17–19]. Any effective mitigation strategy must therefore consider not only invocation patterns but also the probabilistic behavior of container teardown, the economic implications of prewarming, and the long-term sustainability of resource provisioning decisions. Despite recent advances in AI-driven autoscaling and predictive prewarming, existing learning-based approaches typically operate as black-box controllers. They often optimize a single objective (e.g., latency or memory usage) without explicitly modeling the constrained multi-objective nature of serverless environments. Furthermore, many approaches do not provide a formal analytical link between cold-start probability, expected latency, and cost-energy trade-offs. This limits their interpretability, theoretical guarantees, and applicability across diverse deployment scenarios.

To address these limitations, this work formulates cold-start mitigation as a constrained multi-objective stochastic optimization problem. The proposed Unified Cold-Start Optimization Model (UCSOM) explicitly models request arrival dynamics, container teardown behavior, expected latency under cold and warm states, operational cost, and energy consumption within a single analytical framework. This formalization provides a principled foundation for understanding the structural trade-offs inherent in serverless systems and moves beyond empirical benchmarking toward decision-oriented modeling. The overview of the proposed framework showing the overall workflow is presented in Figure 2.

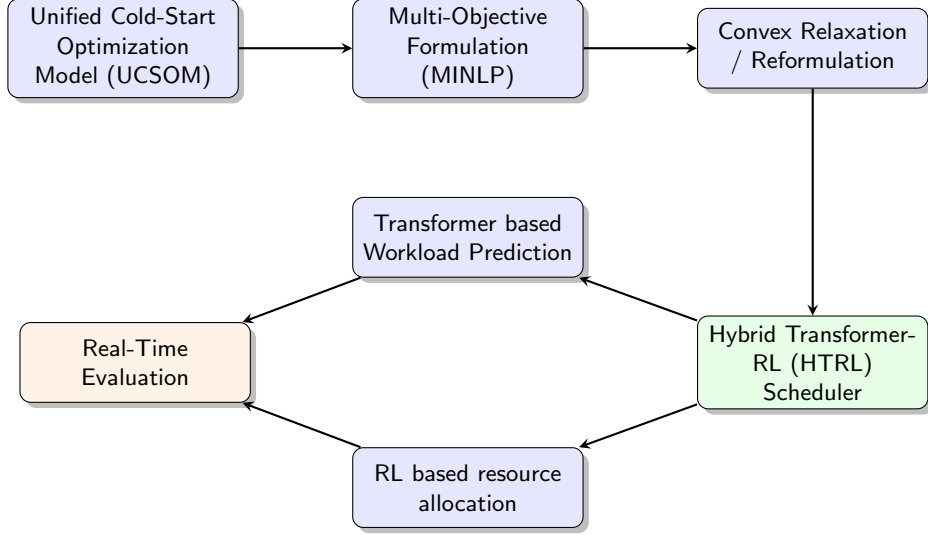


Fig. 2: Overview of the proposed framework

However, directly solving the resulting mixed-integer nonlinear optimization problem in dynamic cloud environments is computationally impractical. Serverless workloads evolve over time, and optimal decisions depend on future demand patterns that are inherently uncertain. Therefore, this paper further bridges theoretical optimization and practical deployment by introducing a hybrid predictive-control framework. By integrating Transformer-based workload forecasting with reinforcement learning-based resource allocation, the static optimization formulation is transformed into a sequential decision-making process under uncertainty. In this way, the learning component is not an isolated heuristic module but an operational approximation of the underlying optimization objective.

Below are the contributions of the proposed work in mitigating the cold start latency in serverless environments.

- A Unified Cold-Start Optimization Model (UCSOM) that formalizes cold-start mitigation as a constrained multi-objective stochastic optimization problem integrating latency, cost, and energy trade-offs.

- A Hybrid Transformer-Reinforcement Learning (HTRL) scheduler that converts static optimization into sequential predictive decision-making.
- Extensive real-world evaluation on AWS Lambda under multiple workload patterns.

The remaining paper is structured as follows: Section 2 presents the required background along with the related works highlighting the gaps in existing works. A uniform cold start optimization model is presented in Section 3 and a hybrid transformer-RL based scheduler algorithm is presented in Section 4. The experimental evaluation in AWS Lambda is presented in Section 5 and Section 6 presents the results of the experiments. The threats to validity are discussed in Section 7 and Section 8 concludes the paper.

2 Related work

Cold start problem is one of the major challenges in serverless environments and lot of research is being carried out in mitigating the latency issue. In this section, different techniques adopted to overcome the cold start problem are presented along with the comparison with our proposed work.

One of the earlier techniques to resolve the cold start latency were mainly based on container runtime reuse and early warming strategies. The authors in [20] have presented a container-based runtime management system a framework that takes advantage of the effective reuse of lightweight containers to address the problem of cold start and long latency of serverless applications. Another similar work proposed by authors in [21], presents a container management architecture to reduce the cold start time based on a modified S2LRU structure, named S2LRU++ with the addition of a third queue. It is deployed in the OpenLambda and tested in AWS and Local VM environment. It was found that the proposed work consumed 50% less memory and 31% average cold start time on average compared to the no-warm method. The authors in [22] proposed a multi-level container reuse which attempts to minimize the startup time of functions in containers with similarities to significantly enhance warm resource use. Because of the difficulty in choosing the most suitable container reuse solutions, a Deep Reinforcement Learning (DRL) based scheduler that can efficiently and effectively solve the issue is developed. The core idea in all these works is to either keep the containers alive or reuse the similar ones. All the works are foundational and infrastructure level and mostly heuristic in nature without formal optimization modeling.

After the advances in artificial intelligence techniques, lot of works were proposed on forecasting based on time-series based prediction. Using the SARIMA model, one of the classical time series forecasting technique, two works were proposed [23, 24] to determine when the incoming request arrives, and therefore to increase or decrease the amount of required containers to ensure that the amount of wasted resources is minimized hence reducing the function launching time. Later, machine learning based techniques [25–27] were proposed where they have used advanced ML techniques based on supervised learning, recurrent neural networks and transformer based models to mitigate the cold start latency. All these techniques predict the next invocation arrival

and prewarm the instances to avoid the latency. However, these works did not focus on the cost and energy bottlenecks caused due to prewarming.

Along with the advanced machine learning techniques, reinforcement learning techniques gained attention with the agents to handle the issue of cold start. The authors in [28] identified the invocation patterns of functions over time based on reinforcement learning approach to determine the suitable time to warm the containers. They have also validated the proposed work on Openwhisk platform and it is observed that there is 12.73% reduction in the memory consumption and 22.65% improvement in the execution invocations of prewarmed containers. Similarly the authors in [29] leverages model-free Q-learning using functional metrics such as response failure rate, CPU utilization and other parameters to proactively warming the functions based on the demand. On the other hand, the authors in [30] proposed a multi-agent model which is based on deep reinforcement learning with intelligent solution for scaling the resources. Along with reducing the cold start latency, their work also offers the flexibility for optimizing the cost to the stakeholders. Though our work also uses RL based technique, these works does not focus explicitly on cost-energy latency.

Based on the above discussion on existing works on mitigating cold start latency, these works focused on time-series prediction, ML based forecasting and RL based adaptive scaling. All these works lack on proper mathematical formal modeling the stochastic cold-start process. In this work, we address this gap by integrating analytical modeling with predictive and sequential control mechanisms.

3 Cold Start Optimization Model

Serverless computing platforms dynamically provision and decommission execution environments in response to incoming requests. While this elasticity improves resource utilization, it introduces cold-start latency caused by container initialization when an inactive function is invoked [31–33]. Cold-start behavior is inherently stochastic, driven by unpredictable request arrivals and provider-specific container lifecycle policies [34]. Consequently, mitigating cold-start latency is not merely a heuristic scheduling problem but a stochastic resource allocation problem involving latency, cost, and energy trade-offs. Existing mitigation techniques primarily rely on empirical prewarming strategies or short-term workload prediction without formalizing the underlying system dynamics. Such approaches often lack theoretical guarantees, fail to capture the probabilistic nature of container idleness, and do not explicitly quantify the multi-dimensional trade-offs between performance and operational overhead.

To address these limitations, a Unified Cold-Start Optimization Model (UCSOM), which models cold-start mitigation as a constrained multi-objective optimization problem grounded in stochastic arrival theory and resource economics. The framework systematically captures: (i) request arrival dynamics, (ii) container lifecycle behavior, (iii) expected latency under cold and warm states, (iv) execution and prewarming costs, and (v) energy consumption due to resource allocation.

This section develops the theoretical foundation of UCSOM. The first step is to define the system model and workload assumptions and then the task is to derive an analytical expression for cold-start probability under Poisson arrivals and formulate the expected latency model. Subsequently, a cost and energy models reflecting real-world serverless pricing and resource consumption is constructed. Finally, we integrate these components into a unified multi-objective optimization formulation and discuss its computational properties. The resulting framework provides a principled foundation for intelligent cold-start mitigation and serves as the analytical backbone for the hybrid learning-based scheduler introduced in the subsequent section.

System Model

Consider a serverless platform hosting N independent functions and for each function $i \in \{1, \dots, N\}$. In serverless systems, requests arrive asynchronously and the arrival patterns are stochastic. To tractable derivation of cold start probability, let $\lambda_i(t)$ be the request arrival rate at time t , the function execution speed μ_i represents the service rate, T_i be the container teardown threshold, M_i represents allocated memory and $x_i \in \{0, 1\}$ be the prewarming decision variable defined as

$$x_i = \begin{cases} 1 & \text{if prewarming is enabled} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

To ensure that the system remains stable and to avoid infinite queue buildup, the system utilization for function i is given by $\rho_i(t) = \frac{\lambda_i(t)}{\mu_i}$ where $\rho_i(t) < 1$.

Cold-Start Probability Model

Assuming that the incoming requests arrive randomly in time following Poisson distribution, the probability that no request arrives during the teardown window T_i is:

$$P_{\text{idle},i} = e^{-\lambda_i T_i} \quad (2)$$

$P_{\text{idle},i}$ indicates that if no requests arrives during the teardown window, the container is destroyed and next request triggers cold start. The cold-start probability is therefore given by $P_{\text{cold},i}$ which is the probability that the next request will experience a cold start.

$$P_{\text{cold},i} = (1 - x_i) e^{-\lambda_i T_i} \quad (3)$$

If prewarming is enabled i.e., $x_i = 1$, then the cold start probability is reduced.

Expected Latency Model

The serverless latency consists of cold start latency and warm invocation latency. Let L_{cold} be the cold-start latency and L_{warm} be the warm-start latency. The expected latency for function i is:

$$L_i = P_{\text{cold},i} L_{\text{cold}} + (1 - P_{\text{cold},i}) L_{\text{warm}} \quad (4)$$

Substituting $P_{\text{cold},i}$:

$$L_i = (1 - x_i)e^{-\lambda_i T_i} L_{\text{cold}} + (1 - (1 - x_i)e^{-\lambda_i T_i}) L_{\text{warm}} \quad (5)$$

It is clear that reduction in cold start directly reduces expected latency.

Cost Model

Serverless pricing typically includes the execution cost, prewarming cost, and memory allocation cost. Let c_e be the cost per request, c_p be the prewarming cost and c_m represent memory cost per unit. Then the total cost for function i is:

$$\text{Cost}_i = c_e \lambda_i + c_p x_i + c_m M_i \quad (6)$$

In serverless computing, higher memory reduces latency but increases the cost and though prewarming reduces the cold start, it adds overhead to the system [35, 36].

Energy Model

To extend the sustainability of the framework, the energy consumption is modeled as:

$$E_i = \delta M_i U_i \quad (7)$$

where δ is the energy coefficient and $U_i = \rho_i T_{\text{obs}}$ is the utilization over observation window. The T_{obs} indicates the observation time window over which system utilization and energy consumption are measured. The energy model reflects increasing focus on green cloud computing as resource over-provisioning leads to carbon cost. The important point to note is that memory allocation consumes energy even when in idle state and prewarming may increase idle energy footprint [37, 37].

Multi-Objective Optimization Problem

Cold start problem cannot be mitigated by just reducing the latency alone because continuous prewarming and allocating required memory may increase the cost and energy consumption [38]. And reducing the cost may lead to cold starts and impact the SLA [39]. Therefore, conflicting objective parameters such as cost, memory, and SLA must be optimized jointly rather than independently which leads to this multi-objective optimization problem. Moreover, this mathematical formulation will serve as the theoretical foundation for the subsequent learning-based scheduler, where the optimization objective is embedded into the reinforcement learning reward function. The unified optimization problem is formulated as given in Equation 8.

$$\min_{x_i, M_i} \sum_{i=1}^N (w_1 L_i + w_2 \text{Cost}_i + w_3 E_i) \quad (8)$$

subject to the following constraints:

SLA Constraint: $L_i \leq L_{\text{SLA}}, \quad \forall i$

Budget Constraint: $\sum_{i=1}^N \text{Cost}_i \leq C_{\text{budget}}$

Memory Bounds: $0 \leq M_i \leq M_{\text{max}}, \quad \forall i$

Binary Decision Constraint: $x_i \in \{0, 1\}, \quad \forall i$

where w_1 , w_1 , and w_1 are the weighting coefficients, L_i is the expected latency of the function i , cost_i indicates the operational cost, E_i is the amount of energy consumed by function i during the observation period, L_{SLA} is the SLA latency threshold, M_{max} and C_{budget} are the maximum allowable memory limit and operational cost for serverless deployment.

This SLA constraint ensures that the expected latency for each function does not violate predefined Service-Level Agreement (SLA) thresholds. It enforces performance guarantees and prevents the optimization from reducing cost at the expense of unacceptable delay. The budget constraint reflects practical cloud deployment limitations, where total operational cost must remain within allocated budget. It introduces global coupling among functions and prevents excessive prewarming or memory allocation. Memory allocation directly influences both execution performance and cost. This bound reflects provider-imposed memory limits and ensures feasible resource provisioning. The prewarming decision is inherently discrete: either a function is provisioned in advance or it is not. This discrete nature introduces combinatorial complexity into the optimization problem.

The resulting optimization problem is classified as a Mixed-Integer Nonlinear Programming (MINLP) problem. The nonlinearity arises from the exponential term in the cold-start probability model and its interaction with the latency formulation. The integer nature of the prewarming decision variable x_i introduces combinatorial complexity. Consequently, finding the globally optimal solution is computationally intractable for large-scale serverless deployments, as MINLP problems are generally NP-hard [40]. This computational complexity justifies the use of relaxation techniques and learning-based approximations, which are explored in the subsequent section.

Convex Relaxation

To enable tractable optimization, the binary constraint $x_i \in \{0, 1\}$ is relaxed and the relaxed problem becomes differentiable and can be solved using gradient-based optimization methods. The final decision is obtained via thresholding x_i again as given in Equation 9.

$$x_i^* = \begin{cases} 1 & \text{if } x_i > \tau \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

where τ is a predefined threshold parameter.

Therefore, this optimization model considers serverless cold-start mitigation as a constrained multi-objective resource control problem grounded in stochastic arrival theory, economic cost modeling, and energy-aware optimization.

4 Hybrid Transformer–RL Scheduler Algorithm

The Unified Cold-Start Optimization model presented in the previous section formulates cold-start mitigation as a constrained multi-objective optimization problem that jointly minimizes latency, operational cost, and energy consumption. While this formulation provides a principled analytical foundation, solving the resulting Mixed-Integer Nonlinear Programming (MINLP) problem in real-time is computationally intractable for large-scale serverless deployments. Furthermore, serverless environments are inherently dynamic, with time-varying workload patterns and stochastic request arrivals that cannot be effectively addressed using static optimization techniques alone [41, 42].

To bridge this gap between theoretical formulation and practical deployment, a Hybrid Transformer–Reinforcement Learning (HTRL) scheduler is proposed that transforms the static optimization problem into a sequential decision-making process under uncertainty. Specifically, the Transformer component models long-range workload dynamics to forecast future request arrival rates, while the reinforcement learning (RL) agent adaptively learns resource allocation policies that approximate the solution of the UCSOM objective. The following subsections describe the workload prediction module, the RL-based control mechanism, and the complete scheduling algorithm.

4.1 Transformer-Based Workload Prediction

Let the historical workload sequence for a serverless function be represented as

$$X_t = \{x_{t-k}, x_{t-k+1}, \dots, x_t\} \quad (10)$$

where x_t denotes the observed request arrival rate at time t , and k represents the look-back window size. The Transformer model is used to learn long-range temporal dependencies and predict the future arrival rate

$$\hat{\lambda}_{t+1} = f_{\text{Transformer}}(X_t) \quad (11)$$

where $\hat{\lambda}_{t+1}$ denotes the predicted request arrival rate at time $t + 1$ and $f_{\text{Transformer}}$ is the nonlinear mapping learned by the Transformer model that converts the historical workload sequence into a predicted future arrival rate. The Transformer architecture leverages self-attention mechanisms to capture periodicity, burst patterns, and long-term workload correlations that are not effectively modeled by short-term recurrent architectures. The predicted arrival rate $\hat{\lambda}_t$ directly impacts the cold start probability and in turn the prediction accuracy improves optimization quality. It converts the framework from reactive scheduling to predictive control.

4.2 Reinforcement Learning-Based Resource Allocation

Based on the predicted workload, a reinforcement learning (RL) agent dynamically determines resource allocation decisions. The state representation, action space and the reward function for the proposed HTRL scheduler are discussed here.

State Representation

To ensure that RL agent has full system visibility, the system state at time t is defined as:

$$S_t = (\hat{\lambda}_t, \rho_t, M_t, P_{\text{cold},t}) \quad (12)$$

where $\hat{\lambda}_t$ is the predicted arrival rate, ρ_t represents system utilization, M_t is the current memory allocation, $P_{\text{cold},t}$ is the estimated cold-start probability. Each of these state variables corresponds to the cost optimization model.

Action Space

The action space is defined as:

$$A_t = \{\text{prewarm}, \text{scale}, \text{memory_adjust}\} \quad (13)$$

These actions correspond to enabling/disabling prewarming, scaling container instances, and adjusting memory allocation. Based on these controllable levers in serverless computing, the RL agent learns how to adjust the optimization variables dynamically.

Reward Function

To align the RL objective with the multi-objective optimization formulation, the reward at time t is defined as:

$$R_t = -(w_1 L_t + w_2 C_t + w_3 E_t) \quad (14)$$

where L_t represents the latency at time t , C_t is the operational cost, E_t is the energy consumption, w_1, w_2, w_3 are the weighting coefficients. The negative sign ensures that minimizing latency, cost, and energy corresponds to maximizing cumulative reward.

Optimization Objective

Serverless systems have temporal dependencies on aggressive prewarming and underprovisioning. The RL agent aims to maximize the discounted cumulative reward:

$$\max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \right] \quad (15)$$

where π is the policy function and $\gamma \in (0, 1)$ represents the discount factor. This formulation enables the agent to learn adaptive resource control policies that balance immediate latency reduction against long-term cost and energy efficiency.

The hybrid scheduler is presented as an Algorithm 1 to determine the resource allocation decisions.

Algorithm 1 Hybrid Transformer-Reinforcement Learning Scheduler

Input: Historical workload sequence X_t

Output: Resource allocation decision A_t

1. Initialize Transformer parameters θ .
2. Initialize RL policy network parameters ϕ .
3. For each time step t :
 - (a) Observe workload history $X_t = \{x_{t-k}, \dots, x_t\}$.
 - (b) Predict arrival rate:

$$\hat{\lambda}_t = f_{\text{Transformer}}(X_t; \theta)$$

- (c) Construct system state:

$$S_t = (\hat{\lambda}_t, \rho_t, M_t, P_{\text{cold},t})$$

- (d) Select action:

$$A_t = \pi_{\phi}(S_t)$$

- (e) Apply action A_t (prewarm / scale / memory adjust).
 - (f) Observe reward:

$$R_t = -(w_1 L_t + w_2 C_t + w_3 E_t)$$

- (g) Update RL policy parameters ϕ .
-

The HTRL framework operationalizes the theoretical cost optimization model formulation by transforming the static multi-objective optimization problem into a sequential decision-making process under uncertainty. By integrating predictive workload modeling with adaptive reinforcement learning control, the framework enables real-time cold-start mitigation while preserving latency-cost-energy trade-offs.

5 Experimental Evaluation

In order to validate the effectiveness in a actual serverless deployment, different options were explored and AWS Lambda was found to be best suitable for performing the experimental evaluation due to its widespread adoption in both research and industry sectors [43, 44]. As a proprietary serverless computing platform from Amazon, AWS Lambda offers several advantages. Notably, it boasts straightforward setup procedures and is renowned for its ease of use [45]. Furthermore, it offers a rich ecosystem of integrations with other AWS services, facilitating the development of robust and interconnected systems. While AWS Lambda excels in simplicity and scalability, the free-tier version has limitations on customization options compared to other platforms [46].

Lambda function

Lambda functions are cloud-based serverless computing units. They run your code without requiring you to manage servers and are activated by events. They are

affordable for brief spikes in activity because they only charge for execution time. This speeds up the development cycle by allowing the developer to concentrate on the code itself, which is written in multiple languages.

Function description: The function that runs in response to a trigger is defined in the AWS Lambda function description. It is the primary entry point and is called from the `lambda_handler` function. Two parameters are accepted by the `lambda_handler` function: `context`, which provides runtime information like function execution time and environment details, and `event`, which contains data related to the trigger. These parameters allow the function to run in the Lambda execution environment while accessing and processing pertinent input data as given below.

```
import json
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    # Log the received event (payload)
    logger.info(json.dumps({"event": event}))
    # Prepare response
    data = {"message": "Event received successfully"}
    return json.dumps(data)

def calculate(length):
    return length*length
```

Additionally, logging has been enabled with the logging level set to "INFO" using the command `logger.setLevel(logging.INFO)`, meaning that messages with a higher severity level and informational messages will be recorded. The used function is simple: it computes and returns the square of its input using the payload of the event. We differentiate between the user function call and the prewarming function call by looking at the payload.

AWS Deployment and invocation: Since the lambda function will be a component of a test setup, we must have a way to call it when necessary, ideally with a script rather than a human trigger. Therefore, in order to connect to Lambda, we have chosen to implement a command line integration with the local system. The function is triggered by this command on the system terminal. Below is the command used for triggering the function on the system terminal.

```
aws lambda invoke --function-name <function_name> --cli-binary-format
raw-in-base64-out --payload
```

In the next step, a Python script is created to run the above command whenever necessary. The command for connecting to AWS Lambda for invocation is contained in the first part of the Python script, and the function for determining the interval between subsequent invocations is contained in the second part as shown below.

```

import subprocess
import time

def invoke_lambda():
    command = [
        "aws", "lambda", "invoke", "--function-name", "cOLD", "--cli-binary-format",
        "raw-in-base64-out", "--payload", '{"length": 5}', "output.txt"
    ]
    subprocess.run(command)

def main():
    offset = 0
    interval = 360 # Interval in seconds
    num_iterations = 30 # Number of iterations(+1)
    for i in range(1,num_iterations):
        invoke_lambda()
        print(f'Invocation i+1 completed.')
        time.sleep(interval) # Increasing interval with each iteration

if __name__ == "__main__":
    main()

```

We conducted a 1-D grid search on the interval parameter i.e., the gap between subsequent invocations. The Initialization Delay parameter is monitored, which serves as a cold start indicator. After conducting experiments, it is found that the container tear-down time is approximately 300 seconds which indicates that if there is more than 300 seconds between function calls, the container will experience increased cold start latency.

Log-Monitoring

The logs monitoring tool supported in AWS is CloudWatch which has been used to monitor the service, specifically logs insights. Customized queries are constructed using CloudWatch Logs Insights to extract specific fields and data from the logs produced by the system. These queries also enable filtering and analysis of logs based on parameters like timestamps, execution time, billed time, and custom metadata. Logging is configured at the INFO level to ensure comprehensive record of events, context variables, and output results. The final structured log data is used as input for the chosen encoder-decoder model.

Test Setup

There are two separate phases to our test setup in order to validate the time series based prediction in AWS Lambda. The prewarming function is not aware of the different patterns in which users invoke the lambda function during the first phase. The encoder-decoder time-series forecasting model is trained using the logs produced during these invocations as input for our dataset. In situations where prewarming is not used, these logs are also used to obtain performance statistics. The developed model then acts as the prewarming function after the training. In phase two, different payloads are used for prewarming and user invocations. Performance analysis is made easier by this segregation, which makes it possible to distinguish

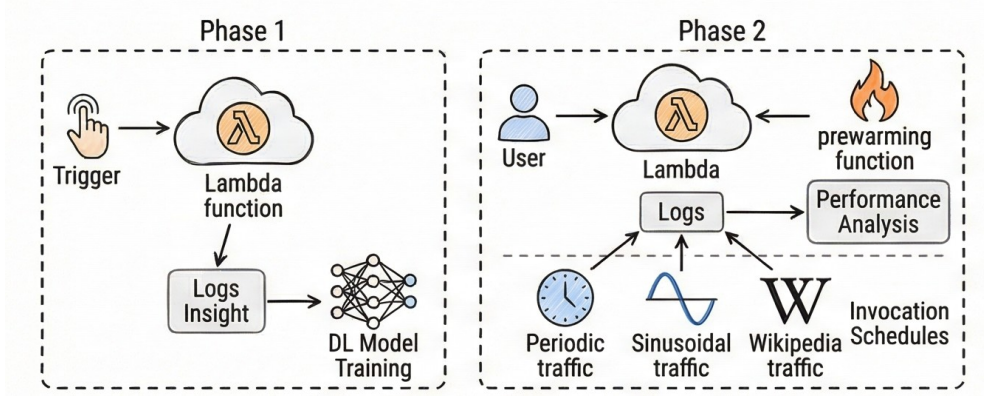


Fig. 3: Test Setup

between different function calls in the logs. Additionally, the prewarming function simulates a function call in this phase by triggering the lambda function. Figure 3 shows a schematic of both the phases. Each of the scenarios listed below requires more than three hours of testing per phase.

- *Periodic Invocation*: In this scenario we have used invocation after a fixed time-period. For testing we have kept the time-period as 6 minutes
- *Sinusoidal invocation*: In sinusoidal invocation we use trigonometric sine function. Further we apply a threshold value of 0.5, i.e. values greater than 0.5 are assigned to a value of 1 indicating an invocation and values below 0.5 indicate user inactivity for that time instance.
- *Wikipedia traffic data trend*: Here we have used the Wikipedia traffic data as user invocation trigger. We use median thresholding here, i.e. values greater than the median are assigned a value of 1 indicating an invocation and values below the median indicate user inactivity for that time instance.

6 Results and Discussion

We will conduct measurements on both average and median execution times, comparing scenarios with and without prewarming. Additionally, we will quantify the extra billing incurred per invocation, an inherent drawback of our approach. Furthermore, we will assess the mishits per invocation, representing the number of failed predictions made by our prewarming function.

6.1 Periodic Invocation Results

In Figure 4, we see the comparison between actual and predicted for periodic invocation for the 3 hour test period. Only Predicted line is visible because the model was able to predict the invocations correctly all the time.

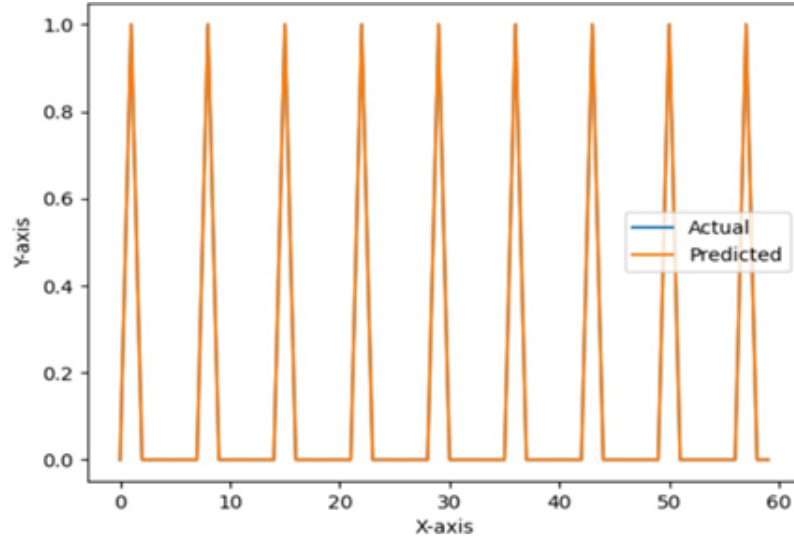


Fig. 4: Actual v/s Predicted for Periodic invocation

As depicted in Table 1, prewarming yields a 15-fold enhancement in average execution time and a 45-fold improvement in median execution time. However, this improvement comes at the cost of an additional billing of 6.25 ms per invocation, which constitutes a drawback. As far as mishits are concerned, its 0 i.e all the predictions were 100% accurate.

Metric	Without Prewarming (in ms)	With Pre-warming (in ms)	Improvement	Extra billing per invocation (average)	Mishits per invocation
Average execution time	95.23	6.21	15×	6.25 ms	0
Median execution time	90.83	2.1	45×	6.25 ms	0

Table 1: Periodic invocation results

6.2 Sinusoidal Invocation Results

The comparison between actual and predicted values for sinusoidal invocation is observed for the 3-hour test period as show in Figure 5. At an instance, a separate blue line is visible which indicates the mishits i.e., the prediction made by the forecasting model is incorrect. It is also observed that a total of 6 mishits has occurred during the 3-hour runtime.

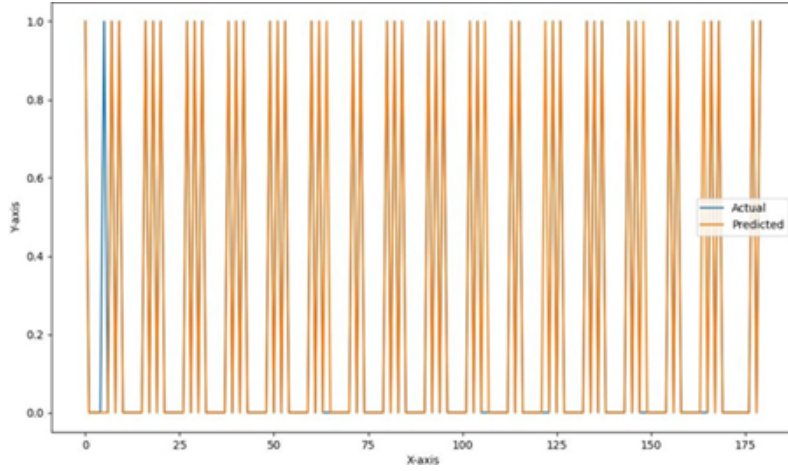


Fig. 5: Actual v/s Predicted for Sinusoidal invocation

As observed in Table 2, the average execution time decreases in the scenario without prewarming. This phenomenon can be attributed to grouped invocations; wherein function invocations occur in clusters with minimal intervals between them. Consequently, this results in the function container remaining in an active state, thus reducing the average execution time. Nevertheless, we see massive improvements after pre-warming, 8x improvement in average time and 20x improvement in median execution time.

Metric	Without Prewarming (in ms)	With Pre-warming (in ms)	Improvement	Extra billing per invocation (average)	Mishits per invocation
Average execution time	65.59	8	8×	6.36 ms	0.03
Median execution time	85.18	4.32	20×	6.36 ms	0.03

Table 2: Sinusoidal invocation results

6.3 Wikipedia trend invocation results

To further test our concept to an extreme, a wiki trend pattern is devised as the dataset that comprises daily aggregate traffic data for Wikipedia websites. We apply median thresholding technique to this dataset, resulting in a binary vector. Subsequently, this vector is utilized for minute-by-minute invocations.

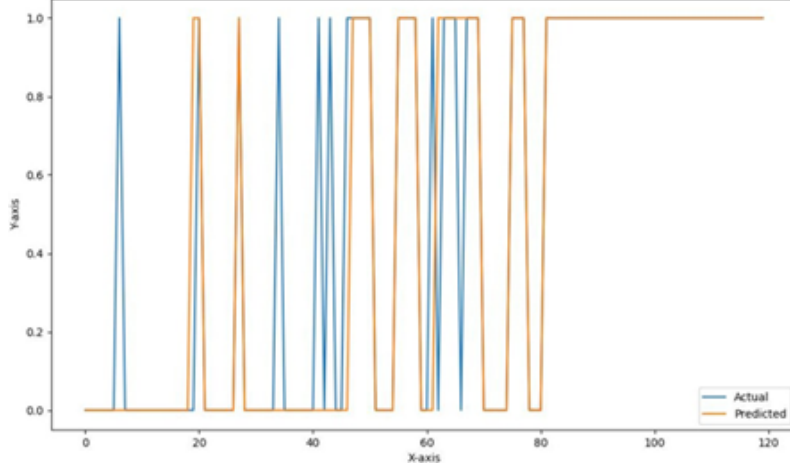


Fig. 6: Actual v/s Predicted for Wiki trend invocation

In Figure 6, it is evident that our model makes incorrect predictions nine times during the entire test duration. Moreover, after the 80-minute threshold, the function encounters a sustained stream of invocations, effectively preventing any occurrence of cold starts. This consistent activity pattern leads to a noteworthy reduction in the average billing per execution, halving the usual amount as illustrated in Table 3. Additionally, the implementation of prewarming demonstrates a substantial reduction, halving the average execution time compared to scenarios without prewarming.

Metric	Without Prewarming (in ms)	With Pre-warming (in ms)	Improvement	Extra billing per invocation (average)	Mishits per invocation
Average execution time	12.12	6.69	2×	3.07 ms	0.06
Median execution time	1.55	1.56	-	3.07 ms	0.06

Table 3: Sinusoidal invocation results

7 Threats to Validity

Numerous factors that could impact on the generalizability of results and dependability could also have an impact on the validity of the research findings.

- *Simulation Fidelity:* The ability of simulation frameworks to replicate the actual serverless environments determines how accurate they are. The Simulated and real performance may differ if resource sharing, container lifecycle management, or invocation patterns are oversimplified using simulators. However the current simulators ignore intricate interactions, which may distort metrics related to latency and resource utilization.

- *Workload Variability*: The Experimental traffic based invocations of periodic, sinusoidal, and Wikipedia used might not always be representative of the variability and uncertainty of real-world serverless workloads. This restricts the applicability of the findings of this work to situations with varying workloads.
- *Platform-Specific Assumptions*: The experimental evaluation in this work is carried out on AWS Lambda only though there exists multiple such platforms like Azure or Knative. The results of this work may not be applicable in other open-source environments due to platform-specific optimizations and limitations.
- *Data Limitations*: Biases may be introduced when ML models are trained using Wikipedia traffic data and artificial invocation patterns. The dataset did not fully capture the more intricate, multi-tenant interactions that are frequently present in real-world serverless workloads.
- *Cost-Latency Trade-off Evaluation*: Although performance improvements such as a 15x reduction in average execution time for periodic invocation are quantified in this paper, the cost overheads such as an extra 6.25 ms of billing per invocation were not thoroughly compared across all platforms or scenarios, which may have understated the financial impact of mitigation strategies.

8 Conclusion

Cold start latency is one of the major challenges faced by serverless computing environments. To address the limitations of existing works, cold start latency problem is formulated as a constrained multi-objective stochastic optimization problem. In specific, a unified cold-start optimization model is proposed which integrates the request arrival rates, container behaviour, expected latency, cost and energy consumption as an optimization model. The formulated model characterizes the cold start problem as mixed integer nonlinear programming problem by providing insights on latency, cost and energy tradeoffs in serverless platforms. Additionally, a hybrid transformer-reinforcement learning scheduler algorithm is proposed to execute the theoretical model in uncertain workloads which transforms the optimization problem into a predictive control problem. On the other hand, to evaluate the time series based prediction model in real-time, the proposed framework is tested on AWS Lambda using periodic, sinusoidal and wiki workload traces. It is observed that the proposed method reduces cold start latency while maintaining cost and energy overhead. In future, as the current model assumes independent functions, incorporating inter-function dependencies and workflow-aware optimization would improve applicability to complex microservice architectures.

References

- [1] Golec, M., Walia, G.K., Kumar, M., Cuadrado, F., Gill, S.S. and Uhlig, S., 2024. Cold start latency in serverless computing: A systematic review, taxonomy, and future directions. *ACM Computing Surveys*, 57(3), pp.1-36.
- [2] Agarwal, S., Rodriguez, M.A. and Buyya, R., 2024, June. On-demand cold start frequency reduction with off-policy reinforcement learning in serverless computing.

- In International Conference on Computing, Intelligence and Data Analytics (pp. 1-24). Singapore: Springer Nature Singapore.
- [3] Kim, Y., Kim, B., Song, T. and Ko, H., 2026. Neighbor-aware shared container instance warming framework for serverless edge computing. *Future Generation Computer Systems*, 174, p.107986.
 - [4] Gao, T. and Xu, M., 2023. Cost control and efficiency optimization in maintainability implementation of wireless sensor networks based on serverless computing. In *Serverless Computing: Principles and Paradigms* (pp. 219-241). Cham: Springer International Publishing.
 - [5] Ustiugov, D., Petrov, P., Kogias, M., Bugnion, E. and Grot, B., 2021, April. Benchmarking, analysis, and optimization of serverless function snapshots. In *Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems* (pp. 559-572).
 - [6] Lin, P.M. and Glikson, A., 2019. Mitigating cold starts in serverless platforms: A pool-based approach. *arXiv preprint arXiv:1903.12221*.
 - [7] Wen, J., Liu, H. and Jing, L., 2023, October. Modeling preference as weighted distribution over functions for user cold-start recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management* (pp. 2706-2715).
 - [8] Fuerst, A. and Sharma, P., 2021, April. Faas-cache: keeping serverless computing alive with greedy-dual caching. In *Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems* (pp. 386-400).
 - [9] P. Verma, P. Goel, N. Rani, "Priority Based Approach For Pre-Warming and Pre-Baking Functions to Optimize Cold Start in serverless Computing," *None*, 2024. <https://doi.org/10.1109/GlobalAISummit62156.2024.10947999>
 - [10] Behera, R.K., Kumari, A. and Cho, S.B., 2025. Scalable Cold-Start Optimization in Serverless Computing: Leveraging Function Fusion With PanOpticon Simulator. *IEEE Access*.
 - [11] Daw, N., Bellur, U. and Kulkarni, P., 2020, December. Xanadu: Mitigating cascading cold starts in serverless function chain deployments. In *Proceedings of the 21st International Middleware Conference* (pp. 356-370).
 - [12] Agarwal, S., Rodriguez, M.A. and Buyya, R., 2021, May. A reinforcement learning approach to reduce serverless function cold start frequency. In *2021 IEEE/ACM 21st international symposium on cluster, cloud and internet computing (CCGrid)* (pp. 797-803). IEEE.

- [13] Kumari, A., Patra, M.K., Sahoo, B. and Behera, R.K., 2022. Resource optimization in performance modeling for serverless application. *International Journal of Information Technology*, 14(6), pp.2867-2875.
- [14] Mampage, A., 2023. *Autonomous Resource Management for Serverless Computing* (Doctoral dissertation, Ph. D. thesis, School of Computing and Information Systems the University of Melbourne, Melbourne, Australia).
- [15] N.Mahmoudi, H.Khazaei, "SimFaaS: A Performance Simulator for Serverless Computing Platforms," *International Conference on Cloud Computing and Services Science*, 2021. <https://doi.org/10.5220/0010376500230033>
- [16] Akkus, I.E., Chen, R., Rimac, I., Stein, M., Satzke, K., Beck, A., Aditya, P. and Hilt, V., 2018. SAND: towards High-Performance serverless computing. In *2018 USENIX annual technical conference (USENIX ATC 18)* (pp. 923-935).
- [17] Shahrad, M., et al. (2020). Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *USENIX Annual Technical Conference* (pp. 205-218).
- [18] Mishra, S. K., et al. (2023). Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions. *arXiv preprint arXiv:2310.08437*.
- [19] Ebrahimi, A., Ghobaei-Arani, M. and Saboohi, H., 2024. Cold start latency mitigation mechanisms in serverless computing: taxonomy, review, and future directions. *Journal of systems architecture*, 151, p.103115.
- [20] Suo, K., Son, J., Cheng, D., Chen, W. and Baidya, S., 2021, September. Tackling cold start of serverless applications by efficient and adaptive container runtime reusing. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)* (pp. 433-443). IEEE.
- [21] Solaiman, K. and Adnan, M.A., 2020, April. WLEC: A not so cold architecture to mitigate cold start problem in serverless computing. In *2020 IEEE international conference on cloud engineering (IC2E)* (pp. 144-153). IEEE.
- [22] Zhou, A.C., Huang, R., Ke, Z., Li, Y., Wang, Y. and Mao, R., 2024, May. Tackling cold start in serverless computing with multi-level container reuse. In *2024 IEEE international parallel and distributed processing symposium (IPDPS)* (pp. 89-99). IEEE.
- [23] Jegannathan, A.P., Saha, R. and Addya, S.K., 2022, June. A time series forecasting approach to minimize cold start time in cloud-serverless platform. In *2022 IEEE International black sea conference on communications and networking (BlackSeaCom)* (pp. 325-330). IEEE.

- [24] Lin, Y., 2024, April. Minimizing Cold Start Time on Serverless Platforms Based on Time Series Prediction Methods. In 2024 IEEE 2nd International Conference on Control, Electronics and Computer Technology (ICCECT) (pp. 996-1001). IEEE.
- [25] Hu, Q., Li, H. and Nikougoftar, E., 2025. Mitigating cold start problem in serverless computing using predictive pre-warming with machine learning. *Computing*, 107(1), p.27.
- [26] Karamzadeh, A. and Shameli-Sendi, A., 2024. Reducing cold start delay in serverless computing using lightweight virtual machines. *Journal of Network and Computer Applications*, 232, p.104030.
- [27] Mouen, A.S.F.M., Zeutouo, J.L. and Tchendji, V.K., 2025. Transformer-Based model for cold start mitigation in FAAS architecture. *arXiv preprint arXiv:2504.11338*.
- [28] Vahidinia, P., Farahani, B. and Aliee, F.S., 2022. Mitigating cold start problem in serverless computing: A reinforcement learning approach. *IEEE Internet of Things Journal*, 10(5), pp.3917-3927.
- [29] Agarwal, S., Rodriguez, M.A. and Buyya, R., 2024, June. On-demand cold start frequency reduction with off-policy reinforcement learning in serverless computing. In *International Conference on Computing, Intelligence and Data Analytics* (pp. 1-24). Singapore: Springer Nature Singapore.
- [30] Mampage, A., Karunasekera, S. and Buyya, R., 2025. A deep reinforcement learning based algorithm for time and cost optimized scaling of serverless applications. *Future Generation Computer Systems*, 173, p.107873.
- [31] Cadden, J., Unger, T., Awad, Y., Dong, H., Krieger, O. and Appavoo, J., 2020, April. SEUSS: skip redundant paths to make serverless fast. In *Proceedings of the Fifteenth European Conference on Computer Systems* (pp. 1-15).
- [32] Jafarpisheh, N., Zaferani, E.J., Teshnehlal, M., Karimipour, H., Parizi, R.M. and Srivastava, G., 2021. A deep neural network combined with radial basis function for abnormality classification. *Mobile Networks and Applications*, 26(6), pp.2318-2328.
- [33] Gunasekaran, J.R., Thinakaran, P., Nachiappan, N.C., Kandemir, M.T. and Das, C.R., 2020, December. Fifer: Tackling resource underutilization in the serverless era. In *Proceedings of the 21st International Middleware Conference* (pp. 280-295).
- [34] Chen, Y., Liu, B., Lin, W., Guo, Y. and Peng, Z., 2025. CASR: Optimizing cold start and resources utilization in serverless computing. *Future Generation Computer Systems*, 170, p.107851.
- [35] Huang, C., Huang, Y., Feng, J., Liang, S., Yan, M. and Wu, J., 2026. LACE: Mitigating cold-starts in serverless with a multi-task mixture-of-experts caching.

- [36] McGrath, G. and Brenner, P.R., 2017, June. Serverless computing: Design, implementation, and performance. In 2017 IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW) (pp. 405-410). IEEE.
- [37] Nair, R.M., 2025. Mitigating Cold Start Latency in Serverless Computing through LLM-Driven Optimization (Doctoral dissertation, Universiteit van Amsterdam).
- [38] Ding, Z., Feng, B. and Yu, W., 2025. FELT: Large-Scale Cloud Workload Prediction Through Adaptive Feature-Enhanced and Similarity-Aware Transformer. Tsinghua Science and Technology.
- [39] Chen, J., He, X., Ye, H., Jiang, F., Zhang, T., Chen, J. and Gao, X., 2025. Online Ensemble Transformer for Accurate Cloud Workload Forecasting in Predictive Auto-Scaling. arXiv preprint arXiv:2508.12773.
- [40] Arbat, S., Jayakumar, V.K., Lee, J., Wang, W. and Kim, I.K., 2022, June. Wasserstein adversarial transformer for cloud workload prediction. In Proceedings of the AAAI Conference on Artificial Intelligence (Vol. 36, No. 11, pp. 12433-12439).
- [41] Yu, H., 2021. Faasrank: A reinforcement learning scheduler for serverless function-as-a-service platforms. University of Washington.
- [42] Wang, L., Li, M., Zhang, Y., Ristenpart, T. and Swift, M., 2018. Peeking behind the curtains of serverless platforms. In 2018 USENIX annual technical conference (USENIX ATC 18) (pp. 133-146).
- [43] Lloyd, W., Ramesh, S., Chinthalapati, S., Ly, L. and Pallickara, S., 2018, April. Serverless computing: An investigation of factors influencing microservice performance. In 2018 IEEE international conference on cloud engineering (IC2E) (pp. 159-169). IEEE.
- [44] Zhang, X., He, Q., Fan, H. and Wu, S., 2024, November. Faascale: Scaling microvm vertically for serverless computing with memory elasticity. In Proceedings of the 2024 ACM Symposium on Cloud Computing (pp. 196-212).
- [45] Wu, C. and Liu, D., 2025, February. Solving the Balance Optimization Problem of Makespan and Total Computation Time for Cloud Computing Scheduling Based on NSGA-II. In 2025 International Conference on Digital Analysis and Processing, Intelligent Computation (DAPIIC) (pp. 783-789). IEEE.
- [46] Xu, Z., Zhang, H., Geng, X., Wu, Q. and Ma, H., 2019, December. Adaptive function launching acceleration in serverless computing platforms. In 2019 IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS) (pp. 9-16). IEEE.